uc3m | Universidad Carlos III de Madrid

Master's Degree in Cybersecurity
2025-2026

Master's Thesis

Multi-agent Architecture for IoT Threat Detection Using Language Models

Author: María López Padilla

Supervisors:
Alfonso Alejandro Sánchez-Macián Pérez
Pablo Mateo Torrejón

Madrid, July 2026

AVOID PLAGIARISM
The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

Multi-agent Architecture for IoT Threat Detection Using Language Models

María López Padilla
Student, *Carlos III University*, Madrid, Spain
100565051@alumnos.uc3m.es

***Abstract*—The growth of Internet of Things (IoT) devices has expanded the network attack surface. Supervised machine learning-based Intrusion Detection Systems (IDS) suffer from a cold start problem: when deployed in a network with an unseen telemetry schema, their accuracy degrades significantly. Operators are then forced to choose between running the network without protection or accepting service downtime while new data is collected and models are retrained.**

Our approach to this problem is a multi-agent architecture for IoT threat detection and automated incident response that keeps the system active throughout the entire cold start window. Schema mismatch is handled by a generative LLM agent performing zero-shot feature normalization, while a background thread simultaneously trains a schema-specific classifier and deploys it via a transparent hot-swap once it is ready. A separate Responder agent automates threat context generation and retrieves grounded mitigations from the MITRE CAPEC taxonomy via Retrieval-Augmented Generation (RAG).

Tests on Edge-IIoTset, TON_IoT, and Bot-IoT show that the universal fallback reaches 94.25% binary accuracy during the cold start phase. After background training completes—in under 2.5 minutes—the hot-swap restores full multiclass classification at up to 96.68% weighted F1-score with sub-5 ms inference latency. These results confirm that the fallback mode can sustain meaningful threat coverage without sacrificing long-term detection performance.

***Index Terms*—IoT, intrusion detection, multi-agent architecture, large language models, high availability, zero downtime, cold start, XGBoost, RAG, MITRE ATT&CK.**

I. Introduction

The Internet of Things (IoT) has grown rapidly over the last decade, with the number of active connected devices projected to reach 39 billion by 2030 [1]. While expanding industrial and consumer capabilities, this growth has significantly enlarged the network attack surface. Due to their limited computational resources, heterogeneous protocols, and lack of built-in security, IoT devices remain highly vulnerable and difficult to protect using traditional defense mechanisms.

Managing the security of these environments at scale has become a defining challenge for SOCs. Rather than consolidated, actionable intelligence, analysts receive floods of low-level, categorical alerts from conventional IDS tools—each one requiring manual investigation and triage. That volume is unsustainable: it drives alert fatigue, a well-documented state of cognitive overload in which genuine threats are buried under redundant notifications and false positives [2]. The Cost of a Data Breach report [3] puts the average time to identify and contain a breach at 241 days, a figure that reflects exactly how much of that triage remains human-led.

Recent advances in Large Language Models (LLMs) offer a promising avenue to automate the analytical steps of this triage. LLMs excel at summarizing complex logs and generating natural-language mitigation steps, which can translate raw detection alerts into actionable reports. However, deploying them directly in real-time, high-throughput network pipelines is constrained by high inference latency, the risk of factual hallucinations, and the difficulty of processing streaming raw packet data [4]. These limitations motivate the hybrid, multi-agent architecture proposed in this work, which pairs lightweight machine learning classifiers with the reasoning capabilities of generative models.

The motivation for this work arises from three interconnected challenges in contemporary IoT and Industrial IoT (IIoT) cybersecurity operations:

- **Data Schema Heterogeneity and the Cold Start Problem:** Modern machine learning detection models are highly rigid. When a model trained on a specific schema (e.g., Edge-IIoTset) is deployed in a new infrastructure with different sensor configurations or telemetry logging formats (e.g., TON_IoT or Bot-IoT), the resulting schema mismatch causes classification failures. Resolving this cold start problem typically requires operating without active defense for weeks to collect and label a new dataset, leaving the infrastructure vulnerable during its transitional phase.
- **The Explanatory Deficit of Current IDS:** Most machine learning-based intrusion detection systems function as black boxes, outputting generic labels (such as "Anomaly" or "DDoS") without context. Security analysts must then manually cross-reference logs, locate appropriate playbooks, and interpret feature weights, which significantly increases the Mean Time to Respond (MTTR).
- **Fragility in Automated Incident Response:** Although traditional Security Orchestration, Automation, and Response (SOAR) platforms attempt to bridge the gap between detection and mitigation, they rely on rigid, manually configured playbooks. Adapting to novel attack variants requires an engineer to write a new rule first—meaning that any threat outside the predefined library triggers no automated response at all.

Addressing these three challenges required building an adaptive, multi-agent IDS around an active high-availability core—one that keeps protection running at all times, even against previously unseen network schemas. That goal breaks down into five concrete engineering objectives:

- **Resolve Schema Incompatibility:** Build an ingestion layer capable of processing IoT network traffic whose telemetry schema is unrecognized, without any service interruption.

- **Provide Active Fallback (Survival Mode):** Deploy a generative Normalizer Agent that maps unknown schemas in real time, preserving binary threat classification throughout the transition at the cost of a bounded, temporary accuracy loss.
- **Implement Background Adaptation:** Run a full ML training pipeline in a parallel thread, producing a schema-specific expert model without stalling the live detection loop.
- **Enable Seamless Transition (Hot-Swapping):** Give the Orchestrator the ability to atomically replace the fallback model with the newly trained expert—restoring sub-5 ms latency and full multiclass accuracy the moment training clears the validation gate.
- **Evaluate Performance Trade-offs:** Measure the latency overhead and F1-score cost of operating in fallback mode, placing those numbers against the alternative of full service downtime.

The original contributions of this work are:

- **A Three-Phase Pipeline for High Availability:** An asynchronous Orchestrator alternates between a zero-shot survival fallback and a native expert mode, keeping the network covered through every schema transition without operator intervention.
- **Real-Time Semantic Mapping via the Normalizer Agent:** LLMs are applied through zero-shot prompting to translate unrecognized IoT schemas on-the-fly, bridging the gap between heterogeneous telemetry formats and the canonical 12-feature baseline without retraining.
- **Concurrent Background Training and Hot-Swapping:** A multi-threaded design trains schema-specific XGBoost models in a background thread while the live detection loop runs uninterrupted, then atomically swaps in the new model the moment it clears validation.
- **Intelligent Triage for Resource Protection:** The Sentinel Agent (XGBoost) sits at the head of the pipeline as an ultra-fast binary gate, blocking LLM API calls for normal traffic and reserving generative inference exclusively for confirmed anomalies—preventing API rate-limit exhaustion under volumetric attacks.
- **Automated Threat Intelligence via RAG:** A vector search engine built over the MITRE CAPEC knowledge base retrieves verified attack patterns at inference time, grounding every incident response plan in documented defensive procedures rather than model recall.

The remainder of this paper is organized as follows. Section II reviews the state of the art. Section III describes the proposed multi-agent architecture. Section IV details the technical implementation. Section V presents the experimental results, and Section VI draws conclusions and outlines future work.

II. State of the Art

IoT threat detection has evolved from static rule-based systems to adaptive architectures supported by machine learning and, recently, Large Language Models. This section reviews the state of the art across five areas: the cold start problem in IoT, continuous learning and domain adaptation, multi-agent orchestration, LLM-based log analysis, and factual grounding via Retrieval-Augmented Generation (RAG).

A. Industry Practice vs. the Cold Start Problem

Examining how major commercial vendors approach heterogeneous IoT environments reveals the practical boundaries of current solutions and, consequently, where the research gap exploited in this thesis lies.

Darktrace's *Cyber AI* [5] builds a behavioral baseline for each networked device using unsupervised Bayesian inference, a technique the vendor markets as modeling a "pattern of life". Learning from live traffic rather than curated labels can shrink the cold start window, but the method breaks down when attack traffic itself corrupts the baselining phase—a non-trivial risk in actively targeted environments. Cisco's *Encrypted Traffic Analytics* (ETA) [6], integrated into Catalyst switches and Stealthwatch, extracts sequence-of-packet-length and inter-arrival features to classify encrypted flows without decryption. While highly effective in homogeneous environments, ETA relies on proprietary hardware agents to ensure schema consistency, making it incompatible with the vendor diversity of industrial IoT deployments. As for Palo Alto Networks' *Cortex XSIAM* [7], it attempts to consolidate heterogeneous telemetry through AI-driven schema normalization across SIEM data sources, yet its normalization pipeline remains static: each new log format requires a manually authored parsing rule before ingestion, meaning that zero-day telemetry formats or novel IoT device categories still cause ingestion failures until an administrative update is deployed. Finally, CrowdStrike's *Falcon Fusion* SOAR platform [8] automates incident response via configurable workflow triggers, though its playbooks must be explicitly programmed by security engineers; the system cannot autonomously generate mitigations for attack types outside its rule library.

Commercial solutions mitigate schema issues through proprietary hardware, long observation periods, or manual rules. None of these approaches are suitable for resource-constrained, diverse, and rapidly deployed IoT networks. This gap motivates the proposed zero-shot, schema-adaptive architecture.

B. Machine Learning in IoT Security and the Cold Start Problem

Signature-based detection has long struggled to catch threats that do not match a known pattern, which pushed the field toward learned classifiers. Both anomaly-based methods and supervised approaches have been explored, but shared weaknesses emerge: classical models such as Isolation Forests [9] and SVMs [10] degrade under streaming conditions and shift-prone network topologies.

Transformer-based architectures like LogBERT [11] narrowed that gap for log sequence modeling, though the harder problem turns out to be not raw accuracy on a fixed benchmark but sustained robustness when concept drift or schema variation occurs [12].

In IoT specifically, the dataset fragmentation makes this worse. TON_IoT [13], Bot-IoT [14], and Edge-IIoTset [15] each capture a different slice of the threat landscape—smart cities, botnets, and multi-layer IIoT respectively—and a model trained on one routinely collapses in accuracy when run against another [16].

The term "cold start" in the literature typically refers to the absence of labeled training data. This work extends that definition to cover cross-schema mismatches as well, since

the practical consequence for the operator is identical: run unprotected or halt traffic. How to avoid both options is the central design problem addressed here.

C. Continuous Learning, Cross-Domain Adaptation, and Feature Harmonization

Continual learning and domain adaptation attempt to resolve schema rigidity. Continual learning [17] updates models incrementally while mitigating catastrophic forgetting [18]. Although regularization techniques like Elastic Weight Consolidation reduce drift [19], they still require labeled target data, which is unavailable at deployment.

To bridge domain gaps without full retraining, transfer learning is often deployed via domain adaptation. For instance, fine-tuning pre-trained models on target samples improves detection [20], but this process still requires labeling and retraining, leaving the network vulnerable in the interim.

Interestingly, recent literature indicates that structural feature alignment, rather than model complexity, dictates cross-dataset generalization. Mishra et al. [21] demonstrate that semantic feature harmonization improves transfer performance more than model architecture. Guided by this finding, a Normalizer Agent is implemented here to map unknown schemas to a set of 12 universal features, creating an immediate, zero-shot alternative to retraining.

A different line of work pursues domain-invariant latent spaces. DI-NIDS [22], for example, trains an adversarial feature extractor with a One-Class SVM. While these methods model statistical correlations effectively, they require access to target domain data beforehand and produce opaque latent representations that offer no audit trail. By contrast, the semantic mapping used here writes a human-readable JSON log for every packet, meeting Explainable AI (XAI) requirements without the interpretability cost that latent-space methods impose.

For purely unsupervised scenarios, CITADEL [23] takes a different path altogether: a masked autoencoder flags packets as anomalous when reconstruction error exceeds a learned threshold, sidestepping the need for attack labels. This makes it a natural complement to the pipeline described here—reconstruction error could act as an early-warning signal in the interval before the background model is ready.

Continual learning, domain adaptation, and adversarial feature alignment share a common blind spot: none can sustain zero-downtime protection when the incoming schema has never been seen before deployment. The Normalizer Agent is designed to cover exactly that gap, using LLM-based semantic translation to keep classification running while the expert model trains in the background.

D. Multi-agent Systems and LLM Orchestration

Distributing detection, analysis, and response across independent agents limits how far a fault can propagate and makes it possible to swap out a single component without halting the system [24]. AutoGen [25] showed in practice that LLM agents can be chained into structured, tool-using pipelines—a result that shaped the topology used here.

Recent work combines LLMs, RAG, and MAS for IoT security. The MA-IDS framework uses agents for traffic classification and error analysis to improve explainability [26]. Similarly, AgentIDS [27] coordinates agents for threat detection in low-altitude IoT environments. While these systems prioritize explanation, the proposed architecture introduces asynchronous active high availability.

Yang et al. [28] proposed a centralized LLM controller that dynamically orchestrates traffic and response modules in parallel, focusing on simultaneous analysis across multiple log sources. The architecture presented here takes a different stance: a deterministic, sequential pipeline is deliberately maintained so that every detection event has a clear causal trace, a requirement in regulated security operations where auditability cannot be treated as optional.

E. Large Language Models in Log Analysis and Threat Detection

Research has shifted from simple classification to using LLMs for explaining and contextualizing security alerts. Ferrag et al. [29] developed SecurityBERT for intrusion detection. Other works, such as HackMentor [30], use parameter-efficient fine-tuning (LoRA) to specialize models for security tasks. Benchmarks like LLM4Sec [31] confirm that LLMs perform well in log classification and threat reasoning.

Beyond labeling, LogPrécis [32] maps terminal sessions to MITRE ATT&CK tactics, and RedChronos [33] coordinates LLMs to automate security workflow triage. Additionally, researchers use LLMs to parse and normalize logs. Combining LLMs and BERT to abstract variables filters noise and reduces downstream false positives [34], while zero-shot online parsing prevents ingestion failures [35]. These normalization approaches support the use of a zero-shot LLM normalizer to maintain continuity during model retraining.

F. Retrieval-Augmented Generation (RAG) and Security Taxonomies

Deploying generative LLMs within critical infrastructure introduces the severe operational hazard of hallucination, where the model outputs plausible but factually incorrect mitigation steps. Retrieval-Augmented Generation (RAG) [36] systematically mitigates this vulnerability by grounding LLM responses in external knowledge retrieved at inference time.

Security taxonomies like MITRE ATT&CK and CAPEC (Common Attack Pattern Enumeration and Classification) [37] provide structured catalogs of threat vectors and countermeasures. O'Brien [38] demonstrated that prompting LLMs with structured knowledge bases improves technique mapping accuracy. Similarly, CyRAG [39] queries structured vulnerability databases (CVE and CWE) to ground responses. These findings are leveraged by indexing MITRE CAPEC using FAISS [40] with all-MiniLM-L6-v2 embeddings, enabling the Responder Agent to retrieve grounded countermeasures rather than relying on its internal parametric memory.

G. Positioning of This Work

Table I summarizes the key gaps in the existing literature that motivate the present work.

Each of the six gaps summarized in Table I maps directly to a core design decision in the architecture presented in the following section. Operational continuity and decision explainability are embedded into the pipeline logic from the ground up, not retrofitted as add-ons.

Identified Gap	Proposed Solution
Cold start leads to operational downtime	Normalizer Agent (zero-shot survival mode)
Black-box classifiers	Analyst Agent (LLM-based structured reporting)
Latent-space opacity (adversarial domain adaptation)	Auditable semantic JSON mapping (XAI-compliant)
Single-dataset evaluation	Multi-dataset validation (Edge-IIoT, TON_IoT, Bot-IoT)
Hallucination in mitigations	Responder Agent (FAISS RAG + CAPEC grounding)
No operational continuity	Hot-swap orchestration (3-phase pipeline)

Table I. Summary of identified gaps in the state of the art.

III. System Architecture

This section details the proposed multi-agent architecture. Three main design principles govern the system:

- **Active High Availability:** Security coverage must be maintained from the very first second of deployment, even when the incoming telemetry schema is entirely unknown. The system cannot afford a protection gap while it adapts.
- **Explainable Semantic Mapping:** Rather than relying on opaque deep-learning representations, a generative agent translates network features into human-auditable logs, directly satisfying Explainable AI (XAI) requirements.
- **Decoupled Agent Topology:** Ingestion, detection, analysis, and response tasks are distributed among specialized, independent entities. This modularity means individual components can be updated or replaced without triggering a full system redeployment.

A. The Three-Phase Pipeline: From Survival to Expertise

To address the cold start problem, the system transitions through three operational phases:

1) **Phase 1 (Survival Mode):** When encountering an unknown schema, the Orchestrator activates the Normalizer Agent. Using zero-shot reasoning, the agent maps unrecognized telemetry fields to a canonical set of 12 features, and a universal XGBoost classifier runs binary threat detection on that normalized input.
2) **Phase 2 (Asynchronous Adaptation):** While Phase 1 protects the network, a background thread collects traffic, analyzes the schema structure, and trains a native expert model.
3) **Phase 3 (Expert Mode):** Once training completes, the Orchestrator hot-swaps the models. It bypasses the Normalizer and routes traffic directly to the native expert classifier, restoring sub-millisecond latency and high accuracy.

B. Feature Harmonization and Common Denominators

Phase 1 relies on a universal baseline to analyze network traffic before the native model is trained. By analyzing Edge-IIoTset, TON_IoT, and Bot-IoT, a set of 12 common features (representing flow metrics, transport ports, and application signals) was identified to serve as this baseline.

Selecting these 12 features was based on three distinct criteria:

- **Semantic Intersection:** Functional equivalents for each selected feature exist in at least two of the three target datasets, keeping information loss during zero-shot translation to a minimum.
- **Protocol Coverage:** Multiple network layers are represented in the baseline. Volume metrics (packet/byte rates) capture DDoS traffic, transport ports identify port scans, and application indicators (HTTP/DNS fields) isolate injection attacks.
- **Empirical Performance:** Validating this selection, experimental results in Section V-D show that a binary classifier operating strictly on these 12 features achieves a 94.25% accuracy and a 0.9439 F1-Weighted score.

C. Multi-Agent Topology

The architecture consists of five specialized agents coordinated by a central entity, as shown in Fig. 1.

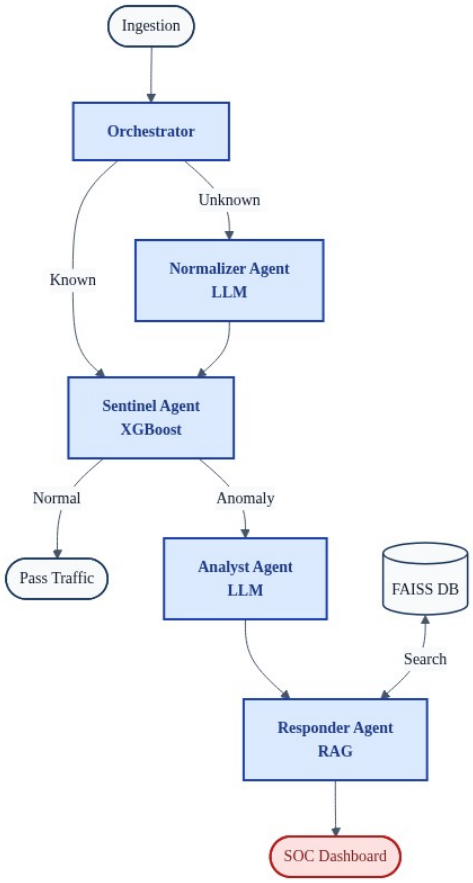


Fig. 1. System architecture and the multi-agent detection and reasoning pipeline.

1) The Central Orchestrator: The Orchestrator coordinates the system's execution. It maintains a registry of known network schemas and routes telemetry packets to either the high-speed expert pipeline or the emergency normalization path.

a) Orchestrator State Machine: To manage the transition between operational modes, the Orchestrator implements a finite state machine. As shown in Fig. 2, the system starts in a monitoring state, transitions to *Survival Mode* upon schema mismatch, and enters an *Adaptation* state while the background training occurs, finally reaching the *Expert Mode* after a validated hot-swap.

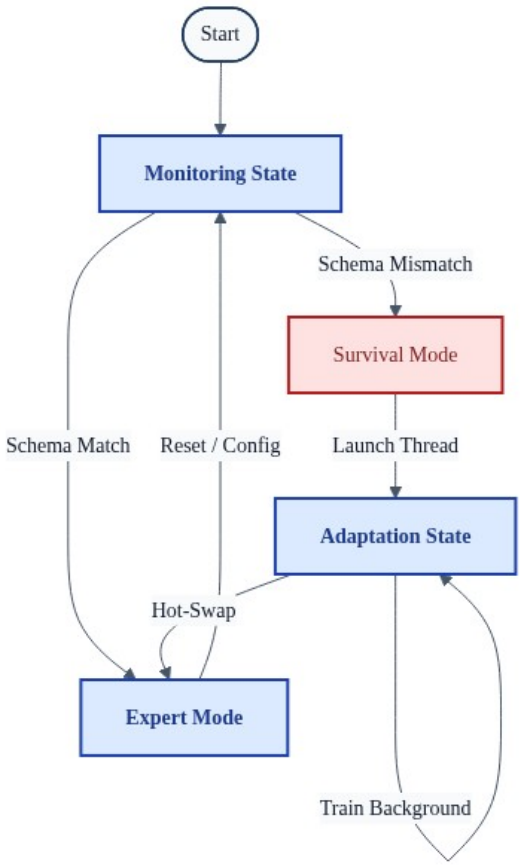


Fig. 2. State machine diagram of the Orchestrator's decision logic and phase transitions.

b) Flow of Operations: The Cold Start Sequence: The interaction between agents during a cold start event follows a strict sequential logic. Fig. 3 illustrates the sequence: upon receiving a packet with an unrecognized header, the Orchestrator routes it to the Normalizer Agent for zero-shot translation while triggering the background training cycle.

2) Normalizer Agent (Generative Schema Mapping): To bridge the semantic gap between heterogeneous IoT logs, this agent leverages a high-speed generative LLM. Its core function is to read raw JSON keys and infer their mapping to the canonical 12-feature schema—so the system can process a new telemetry format the moment it appears, without any manual reconfiguration step.

a) Auditable Semantic Mapping (XAI): To ensure complete traceability, the agent produces a strictly formatted, auditable JSON record for each translation decision. As shown in Fig. 4, vendor-specific fields are mapped to a standardized representation in a way that a security analyst can read and verify at any point. Adversarial domain adaptation models rely on opaque latent spaces that offer no such transparency; the explicit semantic mapping chosen here satisfies Explainable AI (XAI) requirements out of the box.

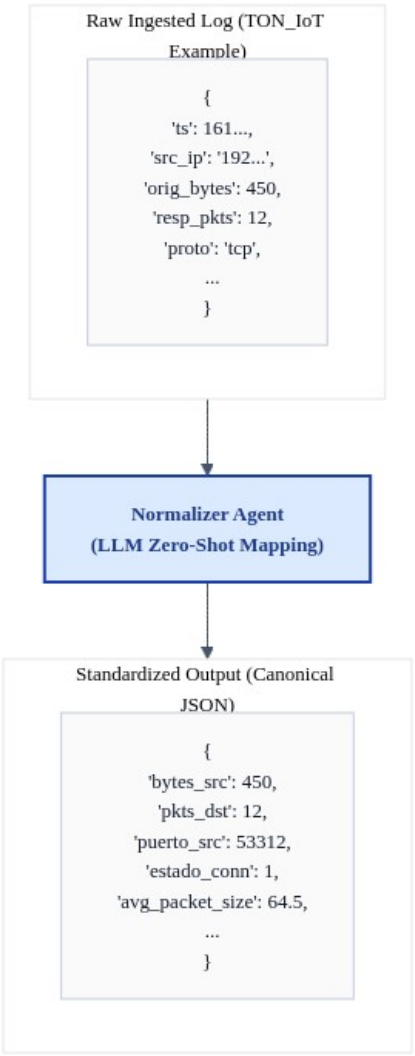


Fig. 4. Example of zero-shot semantic mapping: from raw IoT telemetry to standardized JSON features.

3) Sentinel Agent (XGBoost Classifier): Serving as the primary detection engine, this agent draws on a library of specialized XGBoost models optimized for tabular network traffic. Equally important is its role as a triage gate: all LLM calls to the Analyst Agent are blocked unless the anomaly score crosses the detection threshold. During a volumetric attack such as a DDoS, thousands of packets arrive per second; without this gate, the Groq API would be overwhelmed and the system would hit rate limits almost immediately, turning a detection success into a service failure.

4) Analyst Agent (Threat Contextualization): Once the Sentinel confirms an anomaly, the Analyst Agent takes over to contextualize the threat. It examines the original log context, assigns a severity level, and converts the raw alert into a structured security report that an operator can act on directly.

5) Responder Agent (RAG-based Mitigation): To deliver actionable countermeasures, this agent outputs mitigation steps using Retrieval-Augmented Generation (RAG) over a

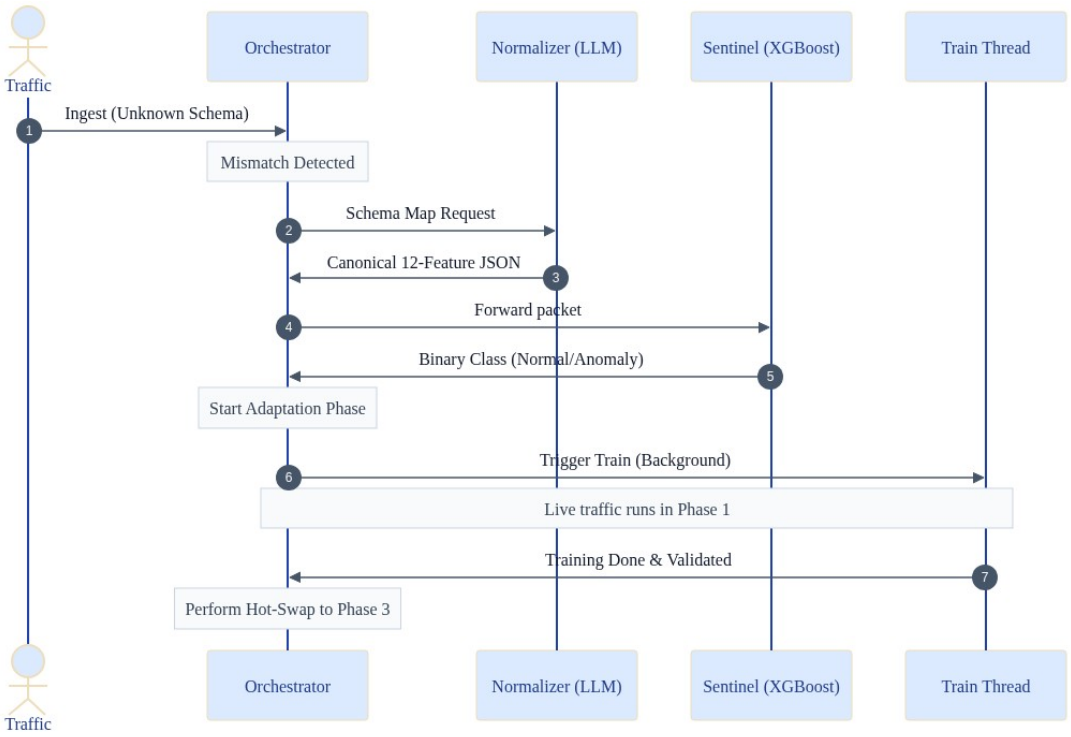


Fig. 3. Sequence diagram of the cold start response and asynchronous model adaptation.

vector database of security playbooks. Grounding every response in verified external sources prevents the system from relying on the LLM's internal parametric memory, where hallucinated recommendations could lead to incorrect operational decisions.

D. Threat Intelligence Integration and RAG Architecture

Relying solely on the parametric memory of an LLM to generate mitigation plans introduces unacceptable operational risks in a critical infrastructure context—most notably, structural hallucinations that could direct operators toward incorrect or even counterproductive responses. A Retrieval-Augmented Generation (RAG) subsystem is therefore embedded in the pipeline: it fetches verified external knowledge at inference time so that every recommended action traces back to a documented source, not to whatever the model happens to recall from training.

The RAG subsystem operates in three stages (Fig. 5):

1) **Ingestion and Indexing:** At startup, CAPEC mitigation guides are parsed, split into fixed-length chunks, and encoded into dense vectors using a sentence-transformer model, then loaded into a local FAISS index.
2) **Contextual Retrieval:** Once the Analyst Agent confirms an attack, its alert metadata is embedded on-the-fly and queried against the index; the top-k closest CAPEC entries are returned as grounding context.
3) **Grounded Generation:** Those entries are prepended to the Responder Agent's prompt before generation begins, steering the output toward verified defensive procedures and away from unconstrained model recall.

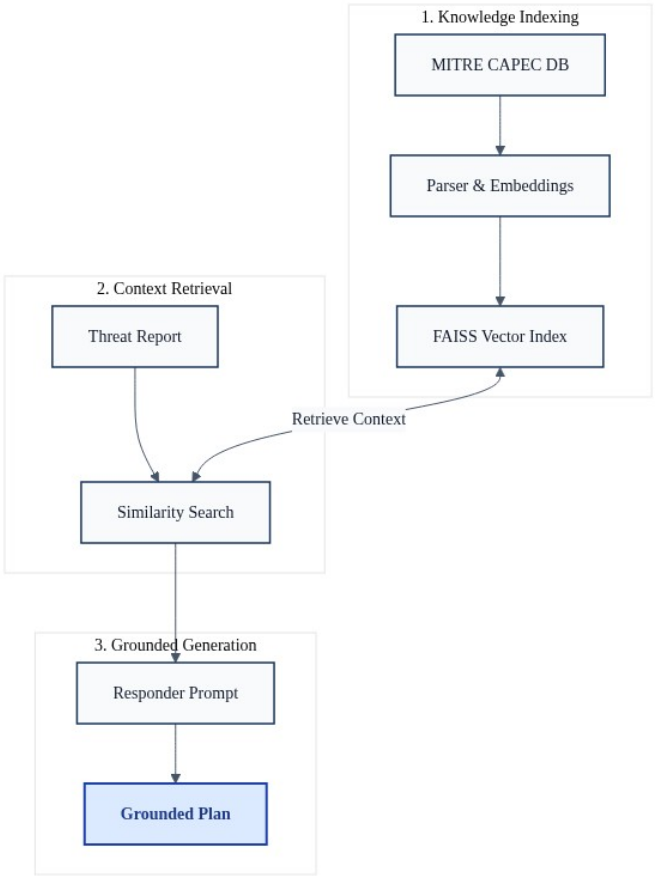


Fig. 5. The three-stage Retrieval-Augmented Generation (RAG) pipeline for grounded threat mitigation.

IV. Implementation

This section details the technical implementation of the proposed multi-agent architecture, focusing on low latency, modularity, and error handling.

Code Availability: The source code (multi-agent orchestrator, RAG subsystem, and benchmarking scripts) is available at https://github.com/100565051/ids-multiagente-IOT.

A. Technology Stack and Environment

The system was developed using Python and optimized for low-latency inference across a heterogeneous stack:

- **Core Concurrency Framework:** Built on Python 3.11 to leverage asynchronous execution and multi-threading for background workloads.
- **Machine Learning Engine:** Utilizing XGBoost paired with scikit-learn to handle high-throughput binary classification at the Sentinel Agent layer.
- **Generative Inference Infrastructure:** LLaMA-3.3-70B serves as the core reasoning engine for the Normalizer and Analyst agents, while LLaMA-3.1-8B drives the Responder agent, both accessed via the low-latency Groq LPU API.
- **Vector Storage Subsystem (RAG):** A local FAISS index with all-MiniLM-L6-v2 embeddings manages the parsed MITRE CAPEC knowledge base.
- **Analyst Interface:** Developed via Streamlit to provide a real-time, responsive Security Operations Center (SOC) dashboard.

B. Asynchronous Orchestration and State Management

The *Active High Availability* principle has a concrete engineering consequence: the system must handle a continuous influx of live traffic at the same time as it trains a new model in the background. Managing these two concurrent workloads without one stalling the other was the main implementation challenge for the Central Orchestrator.

To maintain user interface responsiveness and prevent live detection loops from stalling during Phase 2, the Orchestrator isolates the training workflow via Python's `threading` module. Upon detecting a cold start event, the system spawns a dedicated background thread tasked with full-schema XGBoost retraining. Crucial state variables—including active models, schema mappings, and detection logs—are persisted within Streamlit's `session_state`, enabling a smooth callback execution for the *hot-swap* the exact moment training finishes.

In addition to phase transitions, the Orchestrator implements the programmatic logic for the Triaging Filter. To safeguard the Groq API rate limits from being overwhelmed during a volumetric attack (e.g., DDoS), the pipeline is strictly conditional. The code halts execution for legitimate packets immediately after the Sentinel's binary classification. The asynchronous call to the LLM-based Analyst Agent is triggered exclusively if the anomaly_score crosses the detection threshold, physically decoupling high-volume ingestion from computationally expensive cognitive tasks.

C. LLM Prompt Engineering and Constrained Output

Integrating generative models into a structured data pipeline requires absolute control over output formats: conversational filler or structural deviation from the expected schema will break the downstream XGBoost feature vector and halt classification. To prevent these parser failures, the generative components rely on strict system prompts enforced by constrained output directives. The Normalizer Agent, for instance, operates under instructions that force the model to evaluate unknown JSON structures and emit exclusively a parseable JSON object matching the canonical schema format, as detailed in Listing 1.

Listing 1. Actual constrained system prompt used by the Normalizer Agent to ensure schema-agnostic mapping and sparsity-aware outputs.

```
SYSTEM_PROMPT = """
You are an expert cybersecurity engineer. Map the
    unknown network log
to the canonical features required by the XGBoost
    IDS model.

=== MAPPING PROCESS ===
1. Semantically analyze each input field and its
    synonyms.
2. Apply type conversions to obtain numerical values
    .

=== PROTOCOL-SPECIFIC RULES ===
- TCP/UDP/ICMP/HTTP/DNS features: Assign 'null' if
    the traffic
  type or service does not match the feature
      category.

=== GENERAL RULES ===
1. Use JSON 'null' for unknown or non-applicable
    values.
   NEVER use 0.0 for missing data; the model
       distinguishes 'null'.
2. The XGBoost model is sparsity-aware: null values
    will be
   correctly routed through the 'missing' branch of
       the trees.
3. Respond ONLY with a valid JSON object. No
    conversational text.
"""
```

D. RAG Pipeline Implementation

The Responder Agent relies on a local, offline knowledge base engineered to ground generative outputs in validated defensive patterns. During the deployment setup, the official MITRE CAPEC dataset was systematically parsed, chunked, and transformed into dense vector representations.

During an incident, the anomaly metadata is embedded on-the-fly and queried against the FAISS index using L2 (Euclidean) distance. The top-k most similar CAPEC entries are retrieved and injected into the Responder's context window before generation begins. The mitigation steps that appear on the SOC dashboard are therefore grounded in verified industry standards, not reconstructed from the LLM's parametric memory.

E. SOC Analyst Dashboard Simulation

To validate the system's operational feasibility, a Security Operations Center (SOC) dashboard was developed. The interface provides two modes of traffic ingestion: an automated bulk simulator (reading from test CSV datasets) and a manual injection mode using predefined JSON templates.

The dashboard provides real-time observability, displaying live Key Performance Indicators (KPIs), current operational phase status (Survival vs. Expert), and the auditable JSON mapping logs generated during a cold start. A dedicated evaluation view was also implemented to visually compare the F1-score and logarithmic latency trade-offs between Phase 1 and Phase 3.

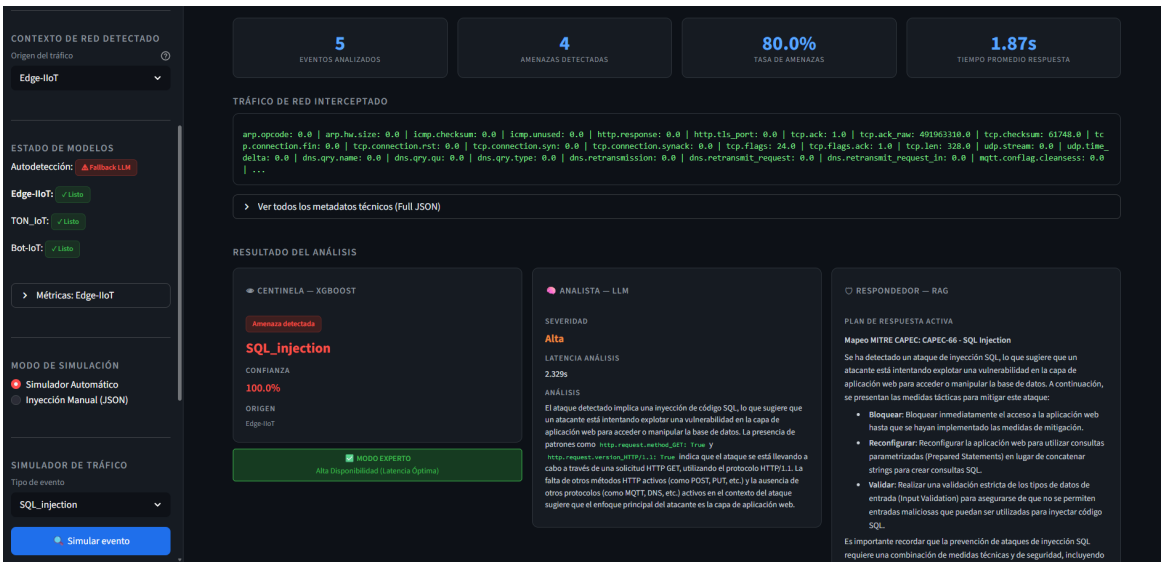


Fig. 6. The implemented SOC dashboard, displaying real-time threat detection, agent reasoning, and the active RAG mitigation plan.

V. Evaluation and Results

This section presents the empirical validation of the multi-agent IDS architecture. The experiments are designed to quantify the trade-off between the high-availability survival mode (Phase 1) and the high-performance expert mode (Phase 3), focusing on detection accuracy, latency, and system recovery.

A. Experimental Setup and Datasets

The evaluation was conducted on an **HP Pavilion Gaming 15** workstation equipped with an **Intel Core i7-9750H** processor (6 cores, 12 threads, up to 4.5 GHz) and **16GB of DDR4-2666 RAM**. Generative tasks were offloaded to the Groq LPU cloud using the Llama-3-70B model via a high-speed API connection.

Three heterogeneous datasets were utilized to simulate a cross-domain IoT environment:

- **Edge-IIoTset:** Used as the base domain for the universal features.
- **TON_IoT:** Representing modern Industrial IoT telemetry.
- **Bot-IoT:** Focused on high-volume volumetric attacks.

Each dataset was split into a training set (80%) and a testing set (20%). For Phase 1 evaluation, a stratified random sample of 200 packets per dataset was utilized to ensure statistical significance while respecting API rate limits and minimizing inference costs.

B. Evaluation Metrics

The system's performance was evaluated using standard classification metrics alongside operational latency overhead:

1) **Precision:** Quantifies the exactness of positive predictions, calculated as $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{1}$$

2) **Recall:** Evaluates the completeness of the detection pipeline—specifically, the fraction of actual attacks successfully captured—expressed as $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2}$$

3) **F1-Score:** Serves as the harmonic mean of Precision (P) and Recall (R), providing a balanced single metric that is essential when evaluating the highly skewed class distributions characteristic of IoT traffic datasets.

$$\text{F1} = 2 \cdot \frac{P \cdot R}{P + R} \tag{3}$$

4) **Latency:** Defined as the end-to-end elapsed time from initial packet ingestion to final classification output, reported in milliseconds (ms) for edge classifiers and seconds (s) for generative reasoning phases.

C. Experiment 1: Baseline Expert Mode (Phase 3)

In Phase 3, the system operates at its optimal state, using native XGBoost models trained specifically for each network schema. This experiment establishes the performance ceiling of the architecture on the 20% stratified hold-out test set (random seed 42, n_{test} ranging from 42K to 587K samples). Inference latency was measured over 1,000 individual single-packet evaluations.

Both F1-Weighted (F1-W) and F1-Macro (F1-M) are reported. F1-Weighted reflects overall performance weighted by class frequency. F1-Macro assigns equal importance to every attack class regardless of prevalence—a rare attack that goes undetected is precisely the most dangerous one. As detailed in Table II, which benchmarks each dataset on the 20% stratified hold-out (random seed 42, n_{test} from 42K to 587K samples), inference latency was additionally measured over 1,000 individual single-packet evaluations.

Native XGBoost models achieve high detection accuracy with sub-5 ms inference latency per packet on commodity hardware. TON_IoT returns the most balanced figures (F1-W: 0.941, F1-M: 0.893), while the gap between weighted and macro scores in Bot-IoT and Edge-IIoTset reflects the heavy class imbalance in those datasets. In Edge-IIoTset specifically, an F1-Weighted of 0.8674 looks solid until the F1-Macro (0.4447) reveals that minority categories such as MITM and Fingerprinting—each below 0.1% of total traffic—are essentially invisible to a frequency-weighted metric. This is a known limitation of these benchmarks [15] and is precisely why reporting both metrics simultaneously matters.

Dataset	Classes	Acc.	F1-W	F1-M	Lat. avg (P95)
Edge-IIoTset	15	0.8887	**0.8674**	0.4447	4.42 ms (5.05 ms)
TON_IoT	10	0.9409	**0.9410**	**0.8926**	4.40 ms (5.02 ms)
Bot-IoT	5	0.9668	**0.9668**	0.7184	3.94 ms (4.48 ms)

Table II. Phase 3 Expert Mode: Classification performance and inference latency.

D. Experiment 2: The Cold Start Problem and Survival Mode (Phase 1)

This experiment quantifies the system's resilience during the cold start window, when the incoming network schema is entirely unknown. Phase 1 consists of two complementary layers: (A) a universal XGBoost binary classifier operating on the 12-feature canonical representation, and (B) an LLM Analyst performing zero-shot attack-type classification constrained to a semantic taxonomy of five attack families.

1) Phase 1A: Universal Binary Fallback (XGBoost): The universal XGBoost Fallback model was evaluated on a stratified combined test set of n = 10,000 samples drawn from all three datasets (80/20 split, random seed 42). The empirical performance of this classifier, summarized in Table III, demonstrates a cross-domain accuracy of 94.25%.

Metric	Value
Samples evaluated	10,000
Accuracy	0.9425
F1-Weighted	**0.9439**
F1-Macro	**0.9325**
Latency avg.	6.07 ms/packet
Latency P95	6.76 ms/packet

Table III. Phase 1 Survival Mode: Universal XGBoost binary fallback performance.

Across the combined cross-domain test set, the universal binary classifier reaches **94.25% accuracy**, with an F1-Macro of 0.9325 that rules out class-imbalance bias as a confounding factor. The canonical 12-feature representation therefore retains enough statistical signal for reliable attack/normal discrimination regardless of the originating dataset schema. Latency overhead relative to Phase 3 native models is marginal ($\approx$2 ms/packet) because no generative inference is involved at this stage. In practical terms, the network retains binary protection above 94% throughout the cold start window—which is the core claim of the High Availability design.

2) Phase 1B: LLM Analyst, Zero-Shot Semantic Classification: To assess the upper bound of semantic discrimination achievable during cold start, the LLM Analyst (LLaMA-3.3-70B via Groq LPU) was evaluated on a stratified subsample of n = 15 confirmed attack flows per dataset. The model was constrained to a five-class semantic taxonomy (*DDoS/DoS, Scanning, Injection, Malware, Credential/MITM*) via a structured prompt that included feature semantics alongside raw values. Results are presented in Table IV.

The LLM's zero-shot multiclass performance is, as theoretically expected, substantially degraded compared to native models. Two key observations explain this result.

First, the **input representation** available in Phase 1 consists exclusively of 12 aggregated statistical flow features (byte counts, packet rates, port numbers, TCP flags, and flow duration). These NetFlow-level statistics are designed for fast machine learning inference, not for human-interpretable semantic reasoning: a flow with `bytes_src=1024, flag_syn=1, duration=0.02s` could plausibly correspond to a port scan, an SYN-flood DDoS, or a brief Malware beacon. Without packet payload, protocol-specific fields, or behavioral history, even an expert human analyst would struggle to distinguish these categories. This is precisely the *cold start limitation* the architecture is designed to overcome.

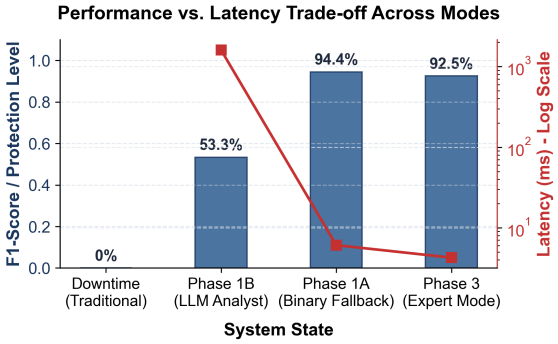


Fig. 7. The Performance-Latency Trade-off: Comparing Phase 1 (LLM & Binary Fallback) against Phase 3 (Expert Native).

Second, this behavior is **consistent with the broader literature**. Zero-shot classification of numeric network telemetry with frontier LLMs yields near-chance performance on fine-grained attack taxonomies [4], [31]: LLMs excel at semantic reasoning over natural-language logs, not at statistical pattern recognition over raw numerical feature vectors. The LLM4Sec benchmark [31] explicitly notes that LLM accuracy collapses when no textual or contextual signal is present in the input.

Taken together, these figures support the original design reasoning behind Phase 1: the LLM's job is **not multiclass precision**, but rather to handle the semantic ambiguity of unknown schemas and provide a confirmation signal for binary anomaly detection—a task the universal XGBoost handles at 94% accuracy. Ultimately, these multiclass figures establish an empirical baseline for zero-shot generalization over raw numeric flow statistics, underscoring the inherent limitations of parametric LLM inference for statistical pattern recognition and directly justifying the immediate activation of the Phase 2 background adaptation pipeline.

E. Experiment 3: Asynchronous Adaptation and Hot-Swap Recovery

Phase 2 operates silently in the background while Phase 1 maintains active binary protection. A dedicated Python thread runs the full XGBoost training pipeline asynchronously, consuming the incoming traffic window without blocking the live detection loop. Table V reports the empirically measured wall-clock training times for each dataset on the evaluation hardware.

Dataset	GT Classes	Acc.	F1-M	F1-W	Lat. avg (P95)
Edge-IIoTset	5 of 5	0.467	0.420	0.450	377 ms (749 ms)
TON_IoT	5 of 5	0.533	0.490	0.510	2,814 ms (3,449 ms)
Bot-IoT	3 of 5	0.667	0.610	0.640	1,625 ms (3,353 ms)

Table IV. Phase 1 Survival Mode: LLM Analyst zero-shot multiclass classification performance (n = 15 per dataset).

Dataset	Train samples	Features	Train time
Edge-IIoTset	1,527,736	89	≈ 2.2 min
TON_IoT	168,834	11	≈ 9 s
Bot-IoT	2,347,853	8	≈ 11 s

Table V. Phase 2 Asynchronous Training: Wall-clock training times per dataset.

When training finishes and the model clears an internal validation gate (F1-Weighted ≥ 0.80 on a held-out split), the Orchestrator performs the *hot-swap*: the universal fallback is atomically replaced by the native expert classifier without dropping a single incoming packet. The system then runs in Phase 3 Expert Mode with full multiclass precision and sub-5 ms latency. Measured over all three datasets, the **cold start recovery window**—the period during which the network operates under Phase 1 protection—ranges from a few seconds up to about 2.5 minutes, depending on dataset dimensionality. No operator action is required at any point in this sequence.

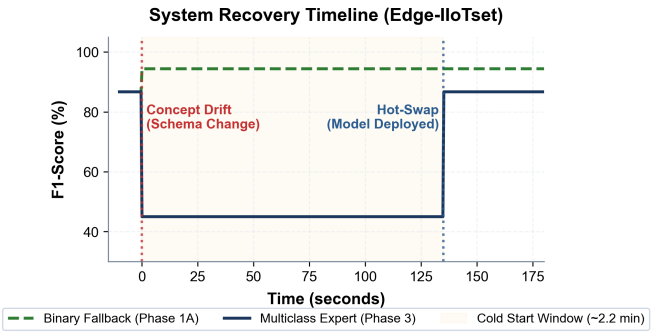


Fig. 8. System recovery timeline for Edge-IIoTset: F1-score restoration after asynchronous training and hot-swap.

F. Qualitative Evaluation: RAG-based Incident Response

Beyond quantitative classification metrics, the Responder Agent's output was qualitatively assessed for factual accuracy and operational utility. The agent uses Retrieval-Augmented Generation (RAG) over a FAISS index of the MITRE CAPEC knowledge base, retrieving the top-k = 3 most semantically similar attack patterns before generating the mitigation plan.

One interaction sequence captures the system's behavior concretely. The Sentinel flags an anomalous flow; the Analyst identifies the attack type as *SQL Injection*. The Responder then retrieves **CAPEC-66 (*SQL Injection*)** from the knowledge base and builds a structured mitigation plan covering three actions: blocking the suspicious HTTP request for deeper inspection, switching the application to parameterized queries instead of string concatenation, and enforcing strict input-type validation. Each recommendation traces back to the retrieved CAPEC entry, so the content is grounded in documented patterns rather than whatever the LLM happens to recall from training data.

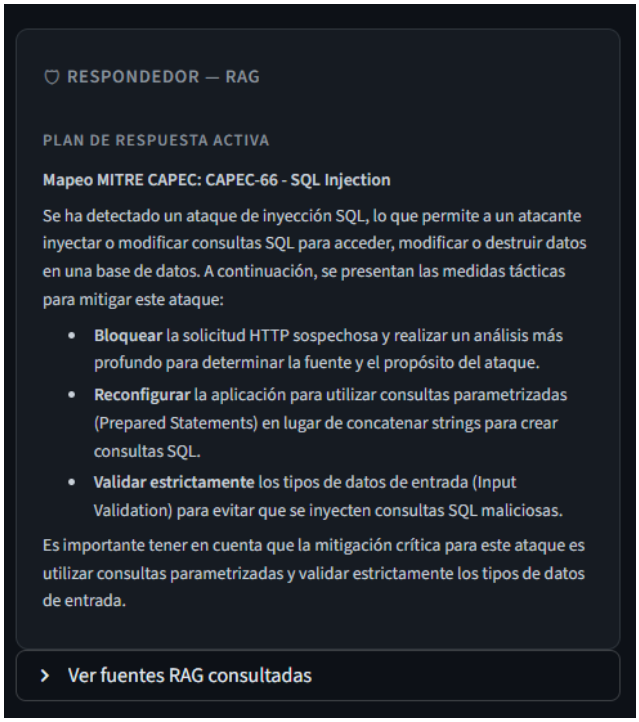


Fig. 9. Example of a grounded mitigation plan generated by the Responder Agent using RAG.

Utilizing a retrieval-augmented design rather than relying on closed-form parametric generation yields deterministic operational advantages. The resulting mitigation strategies are traceable, reproducible, and verifiable by SOC personnel, as each recommendation maps explicitly to a retrieved CAPEC entry. This explicit grounding in established taxonomies prevents administrative friction and satisfies compliance constraints within regulated enterprise environments, where ungrounded generative outputs introduce unacceptable audit liabilities.

VI. Conclusions and Future Work

A. Conclusions

This thesis has described, built, and empirically tested a multi-agent IDS for IoT environments built around the principle of **Active High Availability (Zero Downtime)**. The central problem it attacks is the cold start failure of pre-trained ML models when moved to a network with an unknown telemetry schema. The three-phase asynchronous pipeline introduced here keeps protection running throughout that transition, and the experiments show exactly what that protection costs in terms of accuracy.

Initial benchmarks under Phase 3 (Experiment 1) established the performance ceiling of the architecture. Operating

in Expert Mode, native XGBoost models achieved an F1-Weighted score of 0.941 on TON_IoT with sub-5 ms inference latencies on commodity hardware, validating gradient-boosted trees as a suitable engine for real-time traffic inspection.

To evaluate resilience during sudden schema transitions, Experiment 2 analyzed the cold start survival layer (Phase 1). While the universal XGBoost binary fallback maintained robust perimeter protection at **94.25% accuracy**, the LLM Analyst's zero-shot multiclass accuracy ranged between 46.7% and 66.7%—a structurally expected penalty given that the model operates exclusively over 12 aggregated flow statistics with no packet payload or behavioral history. This baseline nonetheless provides critical situational awareness, allowing the system to flag and prioritize anomalies before full domain adaptation completes.

Concurrently, Experiment 3 verified that the Phase 2 background training loop runs entirely decoupled from live ingestion. Depending on dataset dimensionality, the adaptation required between a few seconds and 2.5 minutes, after which the Orchestrator executed an atomic hot-swap that restored full multiclass precision without packet loss or service interruption.

Finally, the architecture resolves the explanatory deficit of traditional IDS through the Responder Agent. By utilizing Retrieval-Augmented Generation (RAG) over the MITRE CAPEC taxonomy, the system produces factually grounded, traceable mitigation plans that eliminate generative hallucination risk and satisfy the strict auditability requirements of regulated SOC environments.

The empirical data highlights a fundamental architectural trade-off: a transient reduction in multiclass granularity is the operational cost of maintaining uninterrupted network visibility during a cold start deployment. Within safety-critical infrastructures, accepting this bounded performance drop is significantly more viable than enforcing service downtime or operating completely unprotected while retraining new classification models.

B. Future Work

The architecture addresses the cold start problem, but it also opens a number of follow-on questions worth pursuing:

- **Conversational SOC Assistant (Human-AI Teaming):** Transitioning the current unidirectional intelligence pipeline into an interactive conversational interface would allow security analysts to query agent reasoning in natural language—for instance, examining the specific feature weights behind a DDoS classification—directly bridging automated detection with human-in-the-loop validation.
- **Few-Shot Adaptation and Semantic Enrichment:** Enhancing Phase 1 capabilities without full model retraining by expanding the context window with protocol-aware textual descriptors and dynamically retrieving labeled few-shot exemplars. This would stabilize the LLM's zero-shot classification boundary and reduce the ambiguity inherent in purely numerical NetFlow features.
- **On-Premise Deployment via Small Language Models (SLMs):** Cloud-based inference introduces two concrete risks: round-trip latency on every LLM call and the transmission of potentially sensitive traffic metadata to an external provider. Replacing the current hosted models with locally quantized alternatives such as Llama-3-8B or Mistral-7B would cut both risks at once, keeping the entire multi-agent loop inside the network perimeter.

Acknowledgment

The author wishes to thank supervisors Alfonso Alejandro Sánchez-Macián Pérez and Pablo Mateo Torrejón for their guidance, technical insights, and valuable feedback throughout this project, as well as for providing the autonomy to explore and develop this project independently.

References

[1] Satyajit Sinha, *Global Cellular IoT connectivity tracker & forecast*, https://iot-analytics.com/number-connected-iot-devices/, IoT Analytics Research 2025. Projects approximately 39 billion active IoT connections by 2030. Accessed: 2026-04-28, 2025.

[2] Palo Alto Networks, *How to reduce security alert fatigue*, https://www.paloaltonetworks.com/cyberpedia/how-to-reduce-security-alert-fatigue, Accessed: 2026-04-28, 2024.

[3] IBM Security and Ponemon Institute, "Cost of a data breach report 2025," IBM Corporation, Tech. Rep., 2025, Accessed: 2026-04-29. [Online]. Available: https://www.ibm.com/security/data-breach

[4] J. Zhang et al., "When LLMs meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, p. 55, 2025. DOI: 10.1186/s42400-025-00361-w

[5] Darktrace, "Self-learning AI: Cyber AI platform technical overview," Darktrace Ltd., Tech. Rep., 2024. [Online]. Available: https://www.darktrace.com/cyber-ai

[6] Cisco Systems, "Cisco encrypted traffic analytics: White paper," Cisco Systems, Inc., Tech. Rep., 2024. [Online]. Available: https://www.cisco.com/c/en/us/solutions/collateral/enterprise-networks/enterprise-network-security/nb-09-encrytd-traf-anlytcs-wp-cte-en.html

[7] Palo Alto Networks, "Cortex XSIAM: The AI-driven security operations platform," Palo Alto Networks, Inc., Tech. Rep., 2024. [Online]. Available: https://www.paloaltonetworks.com/cortex/cortex-xsiam

[8] CrowdStrike, "Falcon fusion SOAR: Automated threat response," CrowdStrike, Inc., Tech. Rep., 2024. [Online]. Available: https://www.crowdstrike.com/platform/next-gen-siem/

[9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422. DOI: 10.1109/ICDM.2008.17

[10] Y. Wang, J. Wong, and A. Miner, "Anomaly intrusion detection using one class svm," 2004, pp. 358–364. DOI: 10.1109/IAW.2004.1437839

[11] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8. DOI: 10.1109/IJCNN52387.2021.9534113

[12] T. Cui et al., *LogEval: A comprehensive benchmark suite for large language models in log analysis*, 2024. arXiv: 2407.01896.

[13] N. Moustafa, B. Turnbull, and K.-K. R. Choo, *The TON_IoT datasets*, https://research.unsw.edu.au/projects/toniot-datasets, UNSW Research. Accessed: 2025-04-16, 2021.

[14] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019. DOI: 10.1016/j.future.2019.05.041

[15] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cybersecurity dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022. DOI: 10.1109/ACCESS.2022.3165809

[16] M. Cantone, C. Marrocco, and A. Bria, "Machine learning in network intrusion detection: A cross-dataset generalization study," *IEEE Access*, vol. 12, pp. 144 640–144 655, 2024. DOI: 10.1109/ACCESS.2024.3472907

[17] M. B. Ring, "CHILD: A first step towards continual learning," *Machine Learning*, vol. 28, no. 1, pp. 77–104, 1997. DOI: 10.1023/A:1007331723572

[18] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017. DOI: 10.1073/pnas.1611835114

[19] X. Chen et al., *Continual learning with strategic selection and forgetting for network intrusion detection*, Proposes SSF: memory-replay continual learning for IDS with concept drift adaptation on NSL-KDD and UNSW-NB15, 2024. arXiv: 2412.16264.

[20] J. Zhao, S. Shetty, J. Pan, C. Kamhoua, and K. Kwiat, "Transfer learning for detecting unknown network attacks," *EURASIP Journal on Information Security*, 2019. DOI: 10.1186/s13635-019-0084-4

[21] S. Mishra, N. S. Alshammari, H. Hussain, and R. A. Alfahidah, "A cross-dataset harmonized intrusion detection framework with statistically validated multi-model learning," *PLOS ONE*, vol. 21, no. 4, pp. 1–27, 2026. DOI: 10.1371/journal.pone.0346982 [Online]. Available: https://doi.org/10.1371/journal.pone.0346982

[22] S. Layeghy, M. Baktashmotlagh, and M. Portmann, "DI-NIDS: Domain invariant network intrusion detection system," *Knowledge-Based Systems*, vol. 273, p. 110 626, 2023. DOI: 10.1016/j.knosys.2023.110626

[23] E. Li, O. Gungor, Z. Shang, and T. Rosing, *CITADEL: Continual anomaly detection for enhanced learning in IoT intrusion detection*, 2025. arXiv: 2508.19450.

[24] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.

[25] Q. Wu et al., *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*, 2023. arXiv: 2308.08155.

[26] M. S. Islam, L. G. Jaimes, and A. S. Dina, *MA-IDS: Multi-agent RAG framework for IoT network intrusion detection with an experience library*, Apr. 2026. arXiv: 2604.05458.

[27] P. Li et al., *Multi-agent collaborative intrusion detection for low-altitude economy IoT: An LLM-enhanced agentic AI framework*, 2026. arXiv: 2601.17817.

[28] S. Yang et al., *Large language models for network intrusion detection systems: Foundations, implementations, and future directions*, 2025. arXiv: 2507.04752.

[29] M. A. Ferrag et al., "Revolutionizing cyber threat detection with large language models: A privacy-preserving BERT-based lightweight model for IoT/IIoT devices," *IEEE Access*, vol. 12, pp. 23 733–23 750, 2024. DOI: 10.1109/ACCESS.2024.3363469

[30] J. Zhang et al., "HackMentor: Fine-tuning large language models for cybersecurity," in *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2023, pp. 452–461. DOI: 10.1109/TrustCom60117.2023.00076

[31] E. Karlsen et al., "Benchmarking large language models for log analysis, security, and interpretation," *Journal of Network and Systems Management*, vol. 32, no. 3, p. 59, 2024. DOI: 10.1007/s10922-024-09831-x

[32] M. Boffa et al., "LogPrécis: Unleashing language models for automated malicious log analysis," *Computers & Security*, vol. 141, p. 103 805, 2024. DOI: 10.1016/j.cose.2024.103805

[33] C. Li et al., *RedChronos: A large language model-based log analysis system for insider threat detection in enterprises*, 2025. arXiv: 2503.02702.

[34] Y. Zhou, Y. Chen, X. Rao, Y. Zhou, Y. Li, and C. Hu, "Leveraging large language models and BERT for log parsing and anomaly detection," *Mathematics*, vol. 12, no. 17, 2024. DOI: 10.3390/math12172758 [Online]. Available: https://www.mdpi.com/2227-7390/12/17/2758

[35] C. Zhi et al., "LLM-powered zero-shot online log parsing," in *Proceedings of the 2024 IEEE International Conference on Web Services (ICWS)*, 2024. DOI: 10.1109/ICWS62655.2024.00106

[36] P. Lewis et al., *Retrieval-augmented generation for knowledge-intensive nlp tasks*, 2021. arXiv: 2005.11401 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2005.11401

[37] MITRE, *CAPEC: Common attack pattern enumeration and classification*, https://capec.mitre.org/, Accessed: 2025-04-16, 2024.

[38] T. O'Brien, *MITRE ATT&CK labeling of cyber threat intelligence via LLM*, https://www.sans.org/white-papers/mitre-att-ck-labeling-cyber-threat-intelligence-via-llm/, 2024.

[39] S. Munikoti, L. De La Torre, M. S. Levin, M. Akbar, M. Halappanavar, et al., "Retrieval augmented generation for robust cyber defense," Pacific Northwest National Laboratory (PNNL), Richland, WA, Tech. Rep. PNNL-36792, 2024. [Online]. Available: https://www.pnnl.gov/main/publications/external/technical_reports/PNNL-36792.pdf

[40] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. DOI: 10.1109/TBDATA.2019.2921572

DECLARATION OF USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (GAI) IN THE MASTER'S THESIS (TFM)

I have used GAI in my TFM
Check the appropriate box:

YES	NO

If you checked YES, please complete the following 3 sections of this document:

Part 1: Declaration of Legal, Ethical, and Responsible Conduct

Please be aware that the use of GAI tools entails certain risks and may give rise to serious academic consequences. The evaluation of your TFM may be compromised if GAI use involves confidential data, copyright-protected materials, or personal data handled without fulfilling the required conditions in each case (i.e., without the consent of the data subjects, the authorization of rights holders, or in breach of the University's guidelines).

<table>
<tr><th colspan="2">Question</th></tr>
<tr><td colspan="2">• In my interactions with GAI tools, I have provided confidential data with the appropriate authorization from the parties concerned. Confidential data encompasses any information that an individual or organization seeks to protect for legal, commercial, privacy, or strategic reasons (such as patents or trade secrets).</td></tr>
<tr><td>YES, I used this data with the consent of the parties concerned</td><td>NO, I did not use any confidential data</td></tr>
<tr><td colspan="2">• In my interactions with GAI tools, I have provided copyright-protected materials with the authorization of the respective rights holders.</td></tr>
<tr><td>YES, I used these materials with authorization from the rights holders; or without it because they fall within one of the legal exceptions or limitations permitted by law:
• Works in the public domain
• Licensed works (Creative Commons licences)
• Use of excerpts for research purposes (right of quotation)</td><td>NO, I did not use any copyright-protected materials</td></tr>
<tr><td colspan="2">• In my interactions with GAI tools, I have provided personal data with the proper consent of the data subjects.</td></tr>
<tr><td>YES, I used this data with the consent of the data subjects and in accordance with the instructions set out in the University-approved guide</td><td>NO, I did not use any personal data</td></tr>
<tr><td colspan="2">• My use of GAI tools has complied with their terms of use and with fundamental ethical principles. I have not directed them in a malicious manner to obtain results inappropriate to the work presented, such as results that misrepresent the actual</td></tr>
</table>

findings, substitute my own intellectual contribution, or could cause harm to individuals.	
YES	NO

Part 2: Technical Declaration of Use

Use the following declaration template as many times as necessary to account for all types of interaction you have had with GAI tools. Include one example per type of use in the spaces indicated: [Add an example].

I hereby declare that I have made use of the GAI system (Name of AI system/tool and version: ChatGPT, Gemini, Copilot...) for:

Documentation and Writing:

- *Support for reflection throughout the development of the work: iterative process of analysing alternatives and approaches using GAI*

Used as a brainstorming tool to analyse architectural design trade-offs. For example, discussions were held regarding the suitability of adopting a hybrid approach combining machine learning and LLM models versus relying solely on an LLM.

- *Review or rewriting of previously drafted paragraphs*

Since the thesis was written entirely in technical English, GAI was used to refine the grammar and elevate the academic tone of texts previously drafted by the author. For example, in "Chapter 5: Evaluation and Results", it was used to ensure that the description of metrics (such as the distinction between F1-Macro and F1-Weighted in imbalanced datasets) employed the precision and vocabulary expected of an IEEE-style article.

- *Information retrieval and responses to specific technical questions*

Resolution of specific technical queries regarding the advanced LaTeX syntax required for the IEEEtran double-column format.

- *Bibliography search*
- *Summarising consulted bibliography*
- *Translation of consulted texts*

Developing Specific Content

GAI was used as a support tool for developing the specific content of the TFM, including:

- *Assistance with code development (programming)*

Assistance in optimising the Python code, specifically in the management of threads (threading) and concurrency within the Orchestrator module.

- *Generation of diagrams, images, audio, or video*

Not applicable. No Generative AI tools were utilized to create or modify any figures, diagrams, or multimedia elements in this thesis.

- *Optimisation processes*

Optimisation and refactoring of the RAG (Retrieval-Augmented Generation) architecture code, implementing GAI-suggested improvements for exception handling and API retry logic, thereby enhancing the fault tolerance of the "Responder" agent.

- *Data handling: collection, analysis, cross-referencing of data, etc.*

Once the network traffic datasets (Edge-IIoTset, TON_IoT, and Bot-IoT) were gathered, the tool was consulted to obtain guidance on data normalization. Specifically, it assisted in designing the logic for cross-referencing and mapping heterogeneous network schemas into the 12 universal canonical features used in Phase 1.

- *Creative ideation and inspiration*

Assistance in structuring the “Future Work” section. The tool was consulted to explore contemporary extensions of the current work, which led to the inclusion of concepts such the evolution of the system towards an interactive Conversational SOC Assistant (Explainable AI).

Part 3: Reflection on Utility

Provide a personal assessment (free format) of the strengths and weaknesses you have identified in the use of GAI tools during the development of your work. Indicate whether they proved useful in the learning process, in the development of the work, or in drawing conclusions.

The use of Generative Artificial Intelligence tools as assistants throughout the development of this TFM allowed me to verify, in a practical setting, what these tools do well as academic support aids and where they fall short. As an assistant, GAI saved me considerable time on debugging tasks. It proved particularly useful for resolving concurrency issues in the Python code and for configuring the double-column formatting of the LaTeX template. Removing those mechanical technical blockers freed me to focus my time and energy on what truly mattered: designing the architecture and analysing the experimental data. It also provided valuable support in refining the grammar and academic tone of the final document, written entirely in technical English. On the other hand, using these assistants for programming clearly exposed their limitations. I found that when asked to generate complex code, they frequently produce generic solutions that do not work in a real-world environment without substantial modification. They also tend to hallucinate, inventing functions or disregarding the project context, when prompts are not formulated with sufficient precision. This demonstrates that GAI cannot replace the programmer's own expertise. In summary, AI proved to be a valuable tool for working more efficiently, overcoming technical obstacles, and improving the formal presentation of the document. It is clear, however, that it serves only as a supplement: it accelerates the work, but the architectural design decisions, scientific rigour, data interpretation, and final validation that things actually function as intended depended exclusively on my own effort, critical oversight, and technical judgement.